

UNIT 1

Principles of Web Designing & HTML

1. Basic Web Terminology

- **Client:** A client is any computer hardware or software program that connects to a server over a network to request and receive resources, utilizing what the server provides.
- **Resource:** The resources are stored on a server computer and can include data, HTML files, images, audios, videos, etc., which are delivered directly to the client.
- **Internet:** The Internet is a worldwide collection of networks used to connect clients and servers globally across the world.
- **IP (Internet Protocol):** IP stands for Internet Protocol. It is a unique identifier used to discover and establish a connection to a client or server on the network. Every computer connected over a network possesses a unique IP address, acting similarly to a physical residential address used for mailing postal letters to friends and relatives.
- **WWW (World Wide Web):** WWW stands for World Wide Web. It refers to a vast collection of websites or web pages interconnected and accessed globally via the Internet.
- **Web Page:** A document, usually written in HTML, designed to be rendered and displayed to a user by a web browser.
- **Website:** A structured collection of related web pages sharing a common domain name.
- **Web Browser:** A software application (also referred to as a web client) that assists users in requesting and viewing resources from a web server. Common examples include Google Chrome and Mozilla Firefox.
- **URL (Uniform Resource Locator):** Specifying the global address of a web resource (such as an HTML document, audio, or video) over the Internet. It is commonly referred to as a web address. A URL principally contains four structural components:
 1. Protocol
 2. Port number
 3. Domain name
 4. Resource name
- **DNS (Domain Name System):** DNS stands for Domain Name System. It acts as a directory that maps human-readable domain names to their corresponding machine-readable numeric IP addresses. For example, a domain name like `www.sbtet.com` is translated under the hood into an IP address such as `154.197.196.240`.
- **HTTP (HyperText Transfer Protocol):** HTTP stands for HyperText Transfer Protocol. It is the core protocol used to transmit and access resources on the World Wide Web, facilitating requests from a web browser (client) and receiving matching responses from a web server.

2. Introduction to HTML & Basic Page Anatomy

HTML (HyperText Markup Language): HTML stands for HyperText Markup Language. It is a standard markup language composed of various predefined structural elements (markup tags) used to design and format the visual layout of web pages. HTML instructs web browsers exactly how to render and organize textual or multimedia content.

Anatomy of a Web Page: The anatomy of a web page refers to its baseline layout and structural grid. There is no rigid layout requirement, allowing a page to adapt freely to user or client specifications. Generally, a comprehensive web layout includes a page title, URL, header, menu bar, main content panel, left and right navigation sidebars, footers, and standard horizontal/vertical scrollbars.

Basic Structural Tags

- `<h1>` to `<h6>`: Structural heading elements (ranging from highest importance down to lowest).
- `<p>`: Defines paragraph content block.
- `<i>`: Renders italicized text formatting.
- `<b>`: Renders bolded text emphasis.
- `<u>`: Underlines text contents.
- `<strike>`: Draws a cross-through/strikethrough visual line over content.
- `<sup>`: Renders content as a superscript (e.g., mathematical exponents like x^2).
- `<sub>`: Renders content as a subscript (e.g., chemical baselines like H_2O).

Basic HTML Document Skeleton

```
<!DOCTYPE html>
<html>
<head>
  <title>Page Title</title>
</head>
<body>
  <!-- Page Content Goes Here -->
</body>
</html>
```

Note: All HTML files must be saved with a `.html` file extension. To run an HTML file, navigate directly to the storage location on your local machine and open the file using any installed web browser.

3. Structural Page Components & Layout Elements

- **Page Title:** Contained within the `<title>` element inside the page's `<head>` section, this determines the title displayed on the browser tab or window header.
- **Header:** Positioned at the absolute top of the page layout, this area typically houses organizational logos, branding elements, primary navigation menu bars, and integrated search forms.

- **Main Content:** The primary spatial section holding the actual informative content of the web page, including texts, figures, arrays, and structural blocks.
- **Navigation:** Helps users move smoothly from one page to another within a website. This can be organized structurally across four primary positions: Top, Bottom, Left, and Right.
- **Footer:** Standardized structural area pinned to the absolute baseline of a page layout. It commonly contains corporate credits, legal copyrights, site privacy policies, behavioral terms of use, and quick social media shortcuts.
- **Scroll Bars:** Automatically rendered by browsers to allow panning across expansive pages:
 - *Horizontal Scroll Bar:* Pans layout content leftwards or rightwards.
 - *Vertical Scroll Bar:* Moves page viewing focus upwards or downwards.

4. Core Web Page Content Elements

Web pages are composed of several structural elements, each serving a distinct informational role:

1. **Text:** A standard sequence of written characters consisting of headlines, detailed paragraphs, descriptions, or inline annotations.
2. **Images:** Graphic representations (such as photographs, logs, vectors, icons, and illustrations) used to communicate data visually.
3. **Hyperlinks:** Functional anchor markers used to bind different pages together. Clicking a hyperlink transfers user focus seamlessly across destinations:
 - *Anchor Links:* Focus shifts to a separate section within the exact same web page.
 - *Internal Links:* Navigates to an entirely separate page under the same parent website domain.
 - *External Links:* Navigates away to an independent target page hosted on an external website domain.
4. **Frames:** Divides a single browser window into discrete, isolated programmatic boxes capable of displaying independent HTML files simultaneously. Layout frameworks can slice frame borders horizontally, vertically, or via blended matrix views.
5. **Tables:** A cross-gridded display paradigm used to logically convey structured row and column data matrix blocks to clarify correlations between variables.
6. **Media Elements:** Programmatic containers embedding multimedia structures such as audio streams or video playbacks directly inside pages.
7. **Interactive Elements (Forms):** Structured interfaces that gather explicit client-side inputs (like entering a username, checking profile selections, or triggering buttons).

5. Interactive Web Form Design Elements

Web forms gather user inputs using specialized input structures:

1. Text Box

Accepts a single line of standard input text (e.g., full names, mobile numbers, credentials, or email entries).

Syntax: `<label for="name">Name:</label> <input type="text" name="name" id="name">`

2. Checkbox

Permits users to pick multiple complementary answers simultaneously out of a specified selection checklist.

Syntax: `<input type="checkbox" name="argument" id="agreement"> <label for="agreement">I agree</label>`

3. Radio Buttons

Restricts user input selections to a single option within a grouped collection.

Syntax:

```
<p>Gender:</p>
<input type="radio" name="gender" value="Male" id="male"> <label for="male">Male</label>
<input type="radio" name="gender" value="Female" id="female"> <label for="female">Female</label>
```

4. Combo Box (Drop-Down List)

A compact, space-saving list layout where choices slide open dynamically on demand.

Syntax:

```
<label for="branch">Select Branch:</label>
<select name="branch" id="branch">
  <option>CSE</option>
  <option>CSM</option>
  <option>CSI</option>
  <option>IT</option>
</select>
```

5. Text Area

An expansive user textbox input grid intended for receiving multiple lines of text inputs (e.g., feedback fields, multi-line physical mailing addresses, and administrative notes).

Syntax: `<label for="address">Enter Address:</label> <textarea rows="10" cols="20" id="address"></textarea>`

6. Buttons

Functional user controls designed to dispatch inputted information packets directly to a web server, refresh form inputs to zero, or trigger specific scripts.

Syntax: `<button type="submit" value="Submit">Submit</button>`

6. Systematic Lifecycle: Website Creation, Launch & Maintenance

Steps in Building a Website

1. **Requirement Analysis:** Clearly isolate user goals, define features, pinpoint your target user demographic, and list necessary page frameworks.
2. **Planning:** Architect comprehensive navigation flow diagrams, outline domain mapping strategies, and choose production technology stacks.
3. **Designing:** Build mockups, detail visual page flow assets, select interface colors, choose corporate font types, and position images.
4. **Content Creation:** Author descriptive texts, record media audios, produce training videos, and format structured layout matrices.
5. **Deployment (Coding):** Translate creative ideas into clean code using core front-end formats (HTML, CSS, JavaScript) alongside back-end engines.
6. **Testing:** Check for page defects, optimize broken navigation channels, and verify broad multi-browser compatibility.
7. **Deployment (Hosting):** Upload production-ready directories directly to clean web servers to make files public via a registered domain.
8. **Maintenance & Updating:** Frequently update content, patch software bugs, optimize page loading speed, and enforce modern security updates.

Steps in Launching a Website

- Final cross-review of all web pages, media assets, and text logs.
- Comprehensive responsive testing across mobile screens and different desktop viewpoints.
- Registering an available domain name and setting up a secure hosting platform.
- Uploading live file assets through clean server administrative control panels.
- Configuring backend relational data architectures and managing live performance monitoring.

Steps in Maintaining a Website

- Regular textual or asset updates to keep site content fresh.
- Routine hyperlink monitoring to identify and resolve broken URLs or dead-ends.
- Automated background server configuration backups for disaster recovery management.

- Cross-browser compatibility reviews and gathering direct consumer feedback for optimization.

7. Syntax Rules & Document Structures

Rules for Designing HTML Documents

1. Documents should begin with an explicit `<!DOCTYPE html>` version declaration statement.
2. All elements must reside properly nested inside a master root `<html>` wrapper element.
3. Meta structures, styling links, and titles must reside securely within the document's `<head>`.
4. Rendered customer-facing elements belong within the structural `<body>` block container.
5. All standard HTML structural tags are enclosed within angle bracket delimiters (`<tag>`).
6. Closing tags match their opening header counterpart exactly but include a forward slash prefix (`</tag>`). Self-closing standalone structural tags terminate seamlessly via trailing slash notations (`<tag />`).
7. Tag designations and structural element attribute names are case-insensitive; `<html>` and `<HTML>` are treated identically.
8. Attribute settings must be wrapped in straight double quotation symbols (e.g., `<body bgcolor="blue">`).
9. Tags must follow logical nesting hierarchies; outer blocks must remain open until after enclosed child elements have fully terminated (e.g., `<p><b>Hello</b></p>`).
10. Always supply clean fallback `alt` textual logs, explicit widths, and heights on graphical elements (e.g., ``).
11. Document comment tracks are bounded within standard structural comment characters: `<!-- Content Comment Here -->` .

8. HTML Lists, Tables & Advanced Presentation Tags

List Varieties

- **Unordered List (`<ul>`)**: Bulleted display arrangement list format. The structural bullet design variation is managed using the `type` configuration attribute: `disc` (default), `circle` , `square` , or `none` .
- **Ordered List (`<ol>`)**: Systematically cataloged line items tracking sequence via sequential numbering indices. The numbering style is configured using the `type` attribute:
 - `1` : Arabic numeric counters (Default)
 - `I` : Uppercase Roman numerals
 - `i` : Lowercase Roman numerals
 - `A` : Uppercase alphabetical listings
 - `a` : Lowercase alphabetical listings
- **Description List (`<dl>`)**: A data list format matching defined structural terminology headlines (`<dt>`) alongside matching technical description data definition blocks (`<dd>`).

Table Attributes Matrix Reference

Attribute Name	Supported Reference Values	Functional Layout Description
<code>bicolor</code>	Color Name / Hex Code	Defines the background fill color of the table structure.
<code>width</code> / <code>height</code>	Pixels (px) / Percentages (%)	Defines the absolute physical dimensions of the table component.
<code>align</code>	<code>left</code> , <code>right</code> , <code>center</code>	Sets the horizontal alignment of the table on the web page.
<code>border</code>	Integer Values	Sets the physical thickness size of the border gridlines.
<code>cellpadding</code>	Integer Values	Configures structural spatial padding margins between internal data text and cell border walls.
<code>cellspacing</code>	Integer Values	Configures the physical distance spacing separation between individual neighboring cells.
<code>rowspan</code> / <code>colspan</code>	Integer Values	Used inside <code><td></code> or <code><th></code> to merge cells across multiple rows or columns.

Marquee Tag (`<marquee>`)

Creates a scrolling text or image effect across a web page, moving either horizontally or vertically.

- `direction` : Sets the scrolling path (`left` , `right` , `up` , `down`).
- `behavior` : Defines movement dynamics (`scroll` for infinite loops, `slide` to stop at the edge, `alternate` to bounce back and forth).
- `scrollamount` : Sets the movement speed in pixels.

Character Entities Reference

Character entities are structured strings used to display reserved layout characters or invisible markers that cannot be typed directly from a standard keyboard:

- Less-than symbol (`<`): Represented as `<` or `<`
- Greater-than symbol (`>`): Represented as `>` or `>`

3. XML and JS.

⇒ XML:

XML stands for Extensible Markup Language. It is a text based markup language used to store, transport and organize data in a structured and readable format.

→ XML is not for designing web pages like HTML.

→ XML is used to describe data.

→ It allows user to create their own tags.

Key purpose of XML:

- * Data sharing between different systems.

- * Data stored in a structured way.

- * Utilizes for web services like SOAP (Simple Object Access Protocol)

Features of XML:

- * User defined tags

- * Case sensitive

- * Self-descriptive data

- * Easy to read by humans and machines.

Organize data in form of XML.

Data in XML is organized using elements, tags and attributes.

1. XML declaration.

<? XML version="1.0" Encoding="UTF-8"? >

It defines the XML version and character encoding.

<? XML - start of XML

version - widely supported version "1.0"

Encoding UTF (Unicode Transformation Format) - 8 bit

supports all character entities like 0 to 9, A to Z, a to z

and all specialized symbols

2. Root element.

* Every XML document must have one root element.

```
<College>
```

```
⋮
```

```
</College>
```

3. child elements:

Data is organized inside the root element using nested element

```
<college>  
<student id = "40">
```

```
  <Name> P. Nandini </Name>
```

```
  <Roll NO> 24284-CM-040 </Roll NO>
```

```
</student>
```

```
</college>
```

4. Attributes.

It provides additional information can be stored inside tags as attributes.

Syntax: attribute name = "value".

```
<college>
```

```
  <student id = "1">
```

```
    <Name> Nandini </Name> </student>
```

```
  <student id = "2">
```

```
    <Name> Rishma </Name>
```

```
  </student>
```

```
</college>
```

Code --

```
<?XML version = "1.0" encoding = "UTF-8"?>
```

```
<Aritam>
```

```
  <department name = "CME">
```

```
    <HOS> kishore sir </HOS>
```

```
    <strength> 61 </strength>
```

```

    <faculty> 5 </faculty>
</department>
<department name = "ECE">
    <HOS> Muralidhar sir </HOS>
    <strength> 120 </strength>
    <faculty> 8 </faculty>
</department>
<department name = "EEE">
    <HOS> Sagar sir </HOS>
    <strength> 100 </strength>
    <faculty> 6 </faculty>
</department>
<department name = "civil">
    <HOS> Jagan sir </HOS>
    <strength> 60 </strength>
    <faculty> 5 </faculty>
</department>
<department name = "Mech">
    <HOS> Shiva srinivas sir </HOS>
    <strength> 65 </strength>
    <faculty> 6 </faculty>
</department>

```

Rules for Organizing data in XML:

1. Every tag must have a closing tag.
2. Tag must be properly nested.
3. Tag names are case sensitive.

4. One root element is allowed.

5. Attribute values must be in quotation marks.

Significance of Namespace in XML.

A namespace in xml is used to uniquely identify elements and attributes by associating them with a URL and prefix, so that name conflict is avoided, when combining xml document from different sources.

Example:

```
<student>
```

```
<name> Nandini </name>
```

Here name tag could be a student name. Or a book name Or an employee name this creates confusion.

By using namespace we can avoid this conflict.

```
<student>
```

```
<name> Nandini </name>
```

```
<s: name xmlns: "http://edu.in//student">
```

```
</student>
```

Application of XML:

1. Data storage and transport

- store data in a structured and reliable format.
- Platform language independent.

2. Database applications.

Used to import or export data between data bases.

3. Configuration files used in software configuration files.

Example: app configuration files, etc.

4. Document representation used in office documents (word, excel, etc).

5. web servers used in SOAP and REST web services.
(Helps in data exchange between client and server).

6. E-commerce applications used to exchange product, orders, Payment details.

7. Enterprise application used to communicate between business systems.

What is client side scripting?

Client side scripting is a type of scripting where the code runs on the users web browser.

Purpose:

- To make web pages interactive.
- To handle user inputs before sending data to server.

What is server side scripting?

It is a type of scripting where the code runs on the webserver.

Purpose:

- To interact with database.
- To generate dynamic webpages.
- To process data.

Java script:

Java script is a client side scripting language used to create dynamic and interactive webpages.

It runs inside the web browser and works along with html and css.

To print Hello world using Java script:

```

<!DOCTYPE html>
<html>
  <head>
    <title> Java script </title>
  </head>
  <body>
    <script> document. write ("Hello: world") </script>
  </body>
</html>

```

To display message using alert.

Ex:

```

<!DOCTYPE html>
<html>
<head>
  <title> JS </title>
</head>
<body>
  <script> document. write ("Hello")
  <alert = ("meghana")>
</script>
</body>
</html>

```

It will show a pop up message box.

```

<!DOCTYPE html>
<html>
  <head>
    <title> JS </title>
  </head>
  <body>
    <script>
      var college = "Aitam";
      var code = "284";
      document.write ("college name is : " + college + " code : " + code)
    </script>
  </body>
</html>

```

Output :

college name is Aitam code : 284

Arithmetic operations:

```

<!DOCTYPE html>
<html>
  <head>
    <title> Arithmetic operations </title>
  </head>
  <body>
    <script>
      var a = 10;
      var b = 5;
      var sum = a + b;
      var sub = a - b;
      var mul = a * b;

```

```
document.write ("Addition: " + sum);
```

```
document.write ("subtraction: " + sub);
```

```
document.write ("multiplication: " + mul);
```

```
</script>
```

```
</body>
```

```
</html>
```

Output:

Addition: 15

Subtraction: 5

Multiplication: 50

Biggest number using if else statement.

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title> if else </title>
```

```
</head>
```

```
<body>
```

```
<script>
```

```
var a = 10;
```

```
var b = 20;
```

```
if (a > b) {
```

```
document.write ("a is bigger" + a);
```

```
}
```

```
else {
```

```
document.write ("b is bigger" + b);
```

```
}
```

```
</script>
```

```
</body>
```

```
</html>
```

output:

b is bigger so.

Switch case:

```
<!DOCTYPE html>
```

```
<html>
```

```
  <head>
```

```
    <title>Switch Case </title>
```

```
  </head>
```

```
  <body>
```

```
    <script>
```

```
    var vowels prompt("Enter the value:");
```

```
    switch (vowels){
```

```
      case:a
```

```
      document.write("a");
```

```
      break;
```

```
      case:e
```

```
      document.write("e");
```

```
      break;
```

```
      case:i
```

```
      document.write("i");
```

```
      break;
```

```
      case:o
```

```
      document.write("o");
```

```
      break;
```

```
      case:u
```

```
      document.write("u");
```

```
      break;
```

```
      default:
```

```
      document.write("invalid Alphabets");
```

}

</script>

</body>

</html>

Output: Enter the vowel: i

i

For loop:

<!DOCTYPE html>

<html>

<head>

<title> for loop </title>

</head>

<body>

<script>

var n;

var i;

prompt ("Enter the n value:");

for (i=0; i<=n; i++){

document.write ("i:" + i + " ");

}

</script>

</body>

</html>

Output: Enter the n values 5

i=0

i=1

i=2

i=3

i=4

i=5

Functions:

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title> functions </title>
```

```
</head>
```

```
<body>
```

```
<script>
```

```
function display(){
```

```
document.write ("Hii Bantu");
```

```
}
```

```
display ();
```

```
</script>
```

```
</body>
```

```
</html>
```

Output: Hii Bantu.

To display the marks of students using JS.

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title> functions </title>
```

```
</head>
```

```
<body>
```

```
<script>
```

```
function student(){
```

```
var a;
```

```
prompt, ("Enter the marks");
```

```
if (a > 50){
```

```
document.write ("pass");
```

```
else {  
    document.write ("fail");
```

```
}
```

```
}
```

```
student ();
```

```
</script>
```

```
</body>
```

```
</html>
```

output: Enter the marks: 60
Pass.

click-on function:

```
<body>
```

```
<button onclick = "display()" > click me </button>
```

```
<script>
```

```
function display()
```

```
{  
    document.write ("Hii bantu");
```

```
}
```

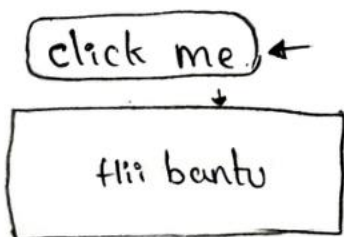
```
display ();
```

```
</script>
```

```
</body>
```

```
</html>
```

Output:



Difference between client side scripting and server side scripting.

Features / Aspect	client side scripting	Server side scripting
Execution place	runs on the user web browser	runs on the web server
main purpose	improves user interaction	perform processing and logic.
access to database	cannot access database directly	can access database
Security	less security because code visible to user.	high security because code was hidden.
response speed	Faster because no server request needed.	slightly slower because server is in processing.
validation	Used for form validation	used for business validation.
code visibility	visible in browser	not visible to the client
dependency on server	No server dependency	fully depends on server
Examples	Java script	PHP, Python, java, etc.

client side scripting languages:

JavaScript, VB script, type script, dart.

Server side scripting languages:

Python, java, PHP, etc.

functions:

A function is a block of code that performs a specific task and runs only when it is called.

Ex: `<script>`

```
function display ()
```

```
{
```

```
document.write ("Hi chinna");
```

```
}
```

```
display ();
```

```
</script>
```

example

1. calling function directly:

The function is called using function name in the script tag itself.

2. calling function using button:

```
<body>
```

```
<button onclick = "display()" > click me </button>
```

```
<script>
```

```
function display ()
```

```
{
```

```
document.write ("Hi nandhu");
```

```
}
```

```
display ();
```

```
</script>
```

```
</body>
```

3. calling function with parameters:

```
<button
```

```
<body>
```

```
<button onclick = "show()"> click me </button>
```

```
<script>
```

```
function show(a,b)
```

```
{
```

```
var sum = a+b;
```

```
document.write ("sum : " + sum);
```

```
}
```

```
show (10,5);
```

```
</script>
```

```
</body>
```

4. Calling function from another function:

```
<script>
```

```
function show()
```

```
{
```

```
display (10,20);
```

```
}
```

```
function display(a,b)
```

```
{
```

```
var sum = a+b;
```

```
document.write ("sum : " + sum);
```

```
}
```

```
</script>
```

5. Calling function when page/page load:

```
<body onload = "display()">
```

```
<script>
```

```
function display(a,b)
```

```
{
```

```
var sum = a+b;
```

```
document.write ("Sum: " + sum);
```

```
}
```

```
display(10, 20);
```

```
</script>
```

```
</body>
```

Arrays: (in Java script):

An array is a special variable used to store multiple values in a single variable.

Example: var numbers = [1, 2, 3, 4];

Syntax: var var-name = [values];

Single dimensional array:

A single dimensional array store values in a single row

To create single dimensional array:

1. declare the array.

Ex: var var-name;

2. Initialize the array.

Ex: var var-name = [values];

var num = [1, 2, 3, 4];

How to access array elements:

Ex Array index start from '0'.

Array index end at 'n-1' for n elements.

Example: var number = [1, 2, 3, 4, 5, 6];

```
document.write (number[0] + "<br>");
```

```
document.write (number[4] );
```

number	1	2	3	4	5	6
	0	1	2	3	4	5

Output: 1

5

Example: To print numbers from 1 to 10. (Even numbers).

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title> Java script </title>
```

```
</head>
```

```
<body>
```

```
<p> Even numbers from 1 to 10 </p>
```

```
<script>
```

```
var num = [1, 2, 3, 4, 5];
```

```
for (var i = 0; i <= 5; i++)
```

```
{
```

```
document.write ( num[i] );
```

```
}
```

```
</script>
```

```
</body>
```

```
</html>
```

Output: numbers from 1 to 10.

1 2 3 4 5 6

Multi-dimensional array:

A multi dimensional array are declared in rows and

columns.

It is mostly used to represent matrix, tables, rows and columns.

Syntax: var arr = [[1, 2, 3],
[4, 5, 6],
[7, 8, 9]];

[[1, 2, 3],
[4, 5, 6],
[7, 8, 9]]

A multi-dimensional array is an array inside another array.

How to access elements:

1. document.write(arr[0]);

→ It will print / give as an output of first row element.

2. document.write(arr[0][0]);

→ It will print first row first element.

Example:

```
<!DOCTYPE html>
```

```
<html>
```

```
  <head>
```

```
    <title> Multi-dimensional array </title>
```

```
  </head>
```

```
  <body>
```

```
    <script>
```

```
      var std = [ ["Nandini", 90],  
                  ["Reshma", 89],  
                  ["Mani", 88],  
                ];
```

```
      document.write ("student : " + std[0][0] + "obtained" +  
                      std[0][1] + "marks");
```

```
    </script>
```

Get ElementById:

The method in javascript used to select or access an html element using id attribute.

Syntax: getElementById () id-value.

Example: <p id="name">Hi </p>

```
  <script>
```

```
    document.getElementById ("name").innerHTML =
```

```
  </script>
```

```
    "Hi Nandini";
```

output - Hi

Hi Nandhu

Example: Getting input value

Enter name: <input type="text" id="value">

<button onclick="display()"> Submit </button>

<script>

function display(){

var sub = document.getElementById("value").value;

alert(sub);

}

</script>

output -

i. Enter name:

ii. Enter name:

← click.

...
 alert message.

* It reads the value entered in the text box

* Displays it using alert().

Global functions provided by JS:

Global functions are classified into several types. They are

1) parseInt()

2) parseFloat()

3) isNaN()

4) isFinite()

5) Eval()

Global functions in JS are built-in functions that can be used anywhere in the program without creating an object.

1. `parseInt()` convert string into an integer.

Example: `<script>
var x = parseInt("500");
var y = 166;
var z = x + y;
document.write(z);
</script>`

var x = "500";
document.write(x);

↓
displays garbage value

Output - 666.

2. `parseFloat()` convert string into float value.

Example: `<script>
var x = parseFloat("30.35");
var y = 5.35
var z = x - y;
document.write(z);
</script>`

Output - 25.00

3. `isFinite()` used to determine whether the specified number is finite or not.

Example: `<script>
document.write(isFinite(100));
document.write(isFinite("Nandhu"));
</script>`

Output - True
False.

4. `isNaN()` checks whether a value is not a number. Returns True or False.

Example: `<script>
document.write(isNaN(105));`

```
document.write (isNaN ("Peshu"));  
</script>
```

output - false
True.

5. eval() evaluates a string as JS code.

Example - <script>

```
var x = prompt ("Enter a number");
```

```
var y = 5;
```

```
var z = eval ("x * y");
```

```
document.write (z);
```

```
</script>
```

output -

Enter a number
10

50

Form handling:

Form handling in JS means accessing validation and processing form data using JS before sending to the server.

JS is mainly used for

1. validation,
 2. check empty fields,
 3. checking email format
 4. preventing form submission if data is incorrect.
- steps

1. create html form

2. Access form elements using JS

3. validate input data. submit the form if valid.

* Accessing form elements are accessed by using

document.getElementById ("id, name").

Example: Enter name: <input type="text" id="name">

Password: <input type="password" id="password">


```

<button onclick = "show()"> Submit </button>
<script>
function show(){
var x = document.getElementById("name").value;
var y = document.getElementById("password").value;
if (x == ""){
document.alertwrite ("Enter name");
return false;
}
if (y == ""){
document.alertwrite ("Enter password");
return false;
}
else { alert ("Form Submitted");
return true;
}
}

```

</script>

Explanation:

1. on submit → calls JS when form is submitted.
2. getElementById() → Access form input field.
3. .value → gets user input.
4. alert() → displays messages.
5. return false → stops form submission.
6. return true → allows form submission.

Math object:

Math object provides mathematical constants and functions.

* It is build-in object in JS.

1. Math.sqrt() → % find square root
2. Math.pow() → % find power

3. Math. random () → It generates random number.
 4. Math. ceil () → Rounds upper no. (Rounds up).
 5. Math. floor () → Rounds lower no. (Rounds down).
- * TO perform mathematical calculations;
- * TO find square root, power and random numbers, etc.

Example:

<script>

document.write (Math.sqrt (121)) + "
;

document.write (Math.pow (5,2)) + "
;

document.write (Math.random () + "
;

document.write (Math.ceil (15.2) + "
;

document.write (Math.floor (15.2) + "
;

</script>

Output - 11

25

16

15

String object:

The string object is used to work with text.

1. String.length () → returns string length.
2. toUpperCase () → It converts text into uppercase.
3. toLowerCase () → It converts text into lower case.
4. charAt () → Returns character.
5. indexOf () → which character present at the index.

where it is used: To manipulate text

To find length

To convert case.

<Script>

document.str = "Apple";

```

document.write (str.length ());
document.write (str.toUpperCase ());
document.write (str.toLowerCase ());
document.write (str.charAt (3));
document.write (str.indexOf (8));
</script>

```

Output: 5

APPLE

apple

1

4

Date object:

Date object work with the date and time.

* TO get current date and time.

* TO calculate / display years, months, etc.

1. new Date () → Gets current date.

2. getFullYear () → Returns / displays year. getFullYear

3. getMonth () → Displays month. getMonth

4. getDate () → Returns current date.

5. getHours () → Displays hours.

For example:

<script>

```

var today = new Date();

```

```

document.write (today.getFullYear ());

```

```

document.write (today.getMonth ());

```

```

document.write (today.getDate ());

```

```

document.write (today.getHours ());

```

</script>

Boolean Object:

- * Boolean Object represents true or false values.
- * It is used in conditional statements and validation.

Syntax: `boolean(1)` → True case

`boolean(0)` → False case

Number Object:

- * Number object is used to work with numerical value.
- `toFixed()`: Formatted decimal
- `isNaN()`: checks not a number

Syntax / Example:

```
<script>
```

```
var x = 28.72;
```

```
document.write("x=" + x.toFixed(2));
```

```
document.write(x.isNaN());
```

```
</script>
```

Output:

x = 28.72

false

- `toString()`: converts numerical value to string

Events in JavaScript

- * An event is an action that occurs in the browser.
- * Events are generated when the user interacts with a webpage.
- * Examples: clicking a button, pressing a key, moving the mouse, submitting a form.
- * JavaScript can detect these events and perform specific actions.
- * There are 3 types of events in JavaScript.

1. Mouse events.

2. Keyboard events.

3. Form and window events.

Mouse events.

1. onclick $\rightarrow$ when user clicks

2. ondblclick $\rightarrow$ double click.

3. onmouseover $\rightarrow$ mouse over element.

4. onmouseout $\rightarrow$ mouse leaves element.

For example:

```
<script button onmouseover="show()"> Submit </button>
```

```
</script>
```

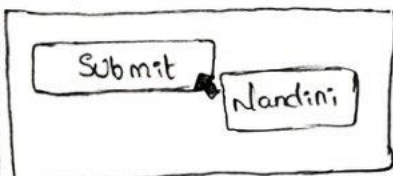
```
function show(){
```

```
document.write("Nandini");
```

```
}
```

```
</script>
```

Output:



Keyboard events:

1. Onkeydown $\rightarrow$ Down key is pressed.

2. Onkeyup $\rightarrow$ Up key is pressed.

3. Onkeypress $\rightarrow$ Any character key is pressed.

Form and window events:

1. Onsubmit $\rightarrow$ when form is submitted.

2. Onload $\rightarrow$ Page loaded.

3. Onunload $\rightarrow$ Page is closed.

4. Onfocus $\rightarrow$ When input field is active.

Why events are used?

- * TO make webpage interactive
- * TO respond to user actions.
- * TO validate forms.

jQuery Comprehensive Study Notes

Complete Reference Guide covering Selectors, DOM Manipulation, Event Handling, and Animations

1. Introduction to jQuery

jQuery is a fast, lightweight, and feature-rich JavaScript library used to simplify HTML document manipulation and event handling. It bridges cross-browser inconsistencies and makes JavaScript programming significantly easier and shorter.

Why is jQuery used?

1. To reduce overall JavaScript code complexity.
2. To simplify DOM (Document Object Model) manipulations.
3. To handle interactive client-side events cleanly.
4. To build intuitive layouts, transitions, and custom visual animations.
5. To perform asynchronous AJAX requests smoothly without browser refreshes.

2. Basic Syntax

The standard format for writing a jQuery statement is:

```
$("#selector").action();
```

- **\$** : The mandatory prefix symbol denoting access to the jQuery library engine.
- **("selector")** : Finds and selects target HTML markup elements utilizing CSS structural notation (IDs, Classes, Elements, etc.).
- **.action()** : Specifies the functional instruction or operational method to invoke over the targeted components.

How to Link an HTML Page with jQuery

1. Download the jQuery framework file locally, or leverage a high-speed CDN endpoint URL.
2. Incorporate the URL asset code snippet inside your document's **<head>** container using a standard script layout:

```
<script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
```

3. Basic Code Examples

Example 1: Displaying "Hello World"

```
<!DOCTYPE html>
<html>
<head>
  <title>Hello World</title>
  <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
</head>
<body>
  <h2 id="value"></h2>

  <script>
    $("#value").text("Hello World");
  </script>
</body>
</html>
```

Expected Viewport Output:

Hello World

Example 2: Addition of Two Numbers

```
<!DOCTYPE html>
<html>
<head>
  <title>Addition</title>
  <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
</head>
<body>
  Number 1: <input type="text" id="no1" /> <br>
  Number 2: <input type="text" id="no2" /> <br>
  <button id="button">Submit</button> <br>
  <h2 id="result"></h2>

  <script>
    $("#button").click(function () {
      var a = parseInt($("#no1").val());
      var b = parseInt($("#no2").val());
      var c = a + b;
      $("#result").text(c);
    });
  </script>
</body>
</html>
```

Dynamic Interaction Test Case:

Number 1: [10]

Number 2: [10]

[Submit Button Click]

Result: 20

Example 3: Click Event Toggle

```
<!DOCTYPE html>
<html>
<head>
  <title>Hello</title>
  <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
</head>
<body>
  <button id="button">Submit</button> <br>
  <h2 id="hi"></h2>

  <script>
    $("#button").click(function() {
      $("#hi").text("Hello World");
    });
  </script>
</body>
</html>
```

4. Core Features of jQuery

- **Lightweight and Fast:** Compact compressed footprint layout executing immediately with zero heavy structural overhead.
- **Simplified Syntax:** Condenses long complex plain vanilla JavaScript phrases into quick, elegant structural configurations.
- **DOM Manipulation (Document Object Model):** Represents structural layout templates natively as simple interactable object trees. Facilitates quick data content queries, real-time styling updates, and asset element attribute additions.
- **Event Handling:** Resolves and tracks multiple distinct dynamic interface tasks safely with minimal setup.
- **Effects and Animations:** Native quick functions bundle built-in transitions like hiding, showing, folding, or opacity sliding properties.
- **Cross-Browser Compatibility:** Operates exactly the same way without programmatic edge cases across Chrome, Safari, Firefox, and Edge engines.
- **AJAX Support:** Drives seamless real-time server lookup exchanges updates completely in the background.

5. jQuery Document Ready Event

The standard `$(document).ready()` function blocks specified code blocks from firing **until the page's entire DOM model has loaded fully and completely** into the browser browser memory engine.

Syntax:

```
$(document).ready(function () {  
    // Operational scripts run here  
});
```

Why use Document Ready?

- Guards scripts from producing unexpected parsing errors on element fields that are not fully built.
- Guarantees reliable user target access across dynamic layout setups.
- Assures enhanced code performance safety.

Note: Attempting to access, modify, or manipulate DOM nodes prior to execution safety completion can cause specific JavaScript statements to fail or report missing parameters.

6. Event Handling in jQuery

Event tracking empowers components to adapt and respond seamlessly to dynamic incoming user interactions.

Event Syntax:

```
$("selector").event(function);
```

Event Classification Matrix

Category	Events Available	Description
1. Mouse Events	<code>click()</code> , <code>dblclick()</code> , <code>mouseenter()</code> , <code>mouseleave()</code> , <code>hover()</code>	Listens to pointing actions and hover movements over targets.
2. Key Events	<code>keydown()</code> , <code>keyup()</code> , <code>keypress()</code>	Captures real-time keyboard adjustments and typing.
3. Form Events	<code>submit()</code> , <code>change()</code> , <code>focus()</code> , <code>blur()</code>	Fires upon field manipulation, data input shifts, or element selections.
4. Document/ Window	<code>ready()</code> , <code>resize()</code> , <code>scroll()</code>	Monitors page loading states, scaling modifications, or offset scrolling positions.

7. jQuery Selectors

Selectors accurately target structural assets from the interactive layout tree structure.

- **Class Selector:** Connects to items via a dot (`.`) syntax identifier.

```
$(".hi").text("Hello");
```

- **ID Selector:** Accesses a specific node uniquely matching a hash (`#`) tag token indicator.

```
$("#Nandhu").text("Hi Nandhu");
```

- **Element Selector:** References raw structural DOM components directly by HTML text keyword tags.

```
$("#button").click(function(){
    $("#h2").text("Hi");
});
```

8. Built-in Effects and Animations

jQuery offers built-in animation parameters designed for user view transformations:

- `hide(speed)` / `show(speed)` : Toggles item presence visibility cleanly.
- `fadeIn(speed)` / `fadeOut(speed)` : Cycles visual parameters gradually through simple alpha transparency updates.
- `fadeToggle(speed)` : Flips and smooths fading properties dynamically based on current status.
- `fadeTo(speed, opacity)` : Adjusts opacity targeting directly into precise point values between `0` and `1`.
- `slideUp(speed)` / `slideDown(speed)` : Shifts content positions vertically via folding collapse methods.
- `slideToggle(speed)` : Seamlessly alternates visual slide heights.

9. DOM Manipulation Functions

`.text()`

Sets or fetches safe string values without parsing HTML code elements.

```
$("#nandhu").text("Nandhi");
```

`.html()`

Fetches or changes raw content containers while processing full executable source elements.

```
$("#hi").html("<b>Hi</b>");
```

`.val()`

Identifies or overrides interactive programmatic user values bound to entry forms or textbox textareas.

```
$("#txt").val("Nandini");
```

`.attr()`

Examines, defines, or updates the value configuration of chosen element layout nodes.

```
$("#picture").attr("src", "image2.jpg");
```

.css()

Directly edits or returns styling layout properties over targeted style structures.

```
$("#me").css("background-color", "pink");
```

WEB SERVERS & SERVER-SIDE SCRIPTING

Complete Typed Lecture & Reference Notes (PHP Core Implementation)

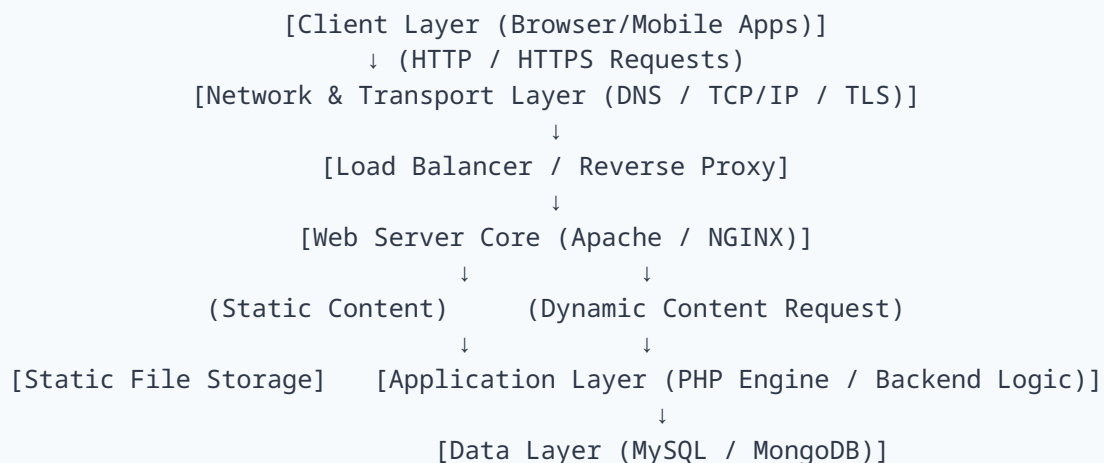
1. Introduction to Web Servers

A **web server** is a foundational system that stores, processes, and delivers web pages to end-users over the internet. This communication is fundamentally governed by standard transport and application layer protocols, primarily **HTTP** (Hypertext Transfer Protocol) and its secure counterpart, **HTTPS**.

Core Concept: Web servers process incoming client requests, locate or dynamically compute the appropriate document/data resource, and issue an accurate network response back to the requester.

2. Architecture of a Web Server

Modern web infrastructure splits server actions into specific tiers to make sure static elements are cleanly separated from application logic and database interactions.



Detailed Architectural Layers:

- 1. Client Layer (Front End):** Comprises web browsers (such as Google Chrome, Mozilla Firefox, Microsoft Edge), mobile applications, and other API services initiating network requests.
- 2. Network and Transport Layer:** Ensures data packets safely and accurately arrive at the destination hosting environment.
 - *DNS*: Resolves user-friendly domain names into standard operational machine IP addresses.
 - *TCP/IP*: Establishes a reliable, connection-oriented network transport stream.

- *TLS/SSL*: Transport Layer Security / Secure Sockets Layer encrypts cleartext data streams for privacy.
3. **Web Server Core**: Represents the heart of the operational system. Its primary responsibilities include:
 - Accepting incoming connection sockets.
 - Parsing raw incoming HTTP requests.
 - Routing requests to the appropriate handler service, serving static files directly, or forwarding dynamic executions.
 - *Internal Components*: Includes the **Request Parser** (reads headers, methods, and parameters), the **Response Generator** (builds standard compliant headers), and the **Connection Handler** (manages concurrency, thread loops, and network sockets).
 4. **Application Layer (Backend Logic)**: Invoked dynamically to handle business processes. It executes script files using software runtimes like PHP, Python, Java, or Node.js, managing logical operations, validation, and authentication checks.
 5. **Data Layer**: Responsible for storage and retrieval. It includes relational/non-relational database management systems (e.g., MySQL, MongoDB) and persistent file stores (local disks or cloud environments).
 6. **Reverse Proxy and Load Balancer**: A reverse proxy receives client requests and passes them along to hidden backend instances. A load balancer optimizes traffic allocation among clusters to scale capacity, lower response latency, and prevent downtime.
 7. **Security Layer**: Works continuously across every tier of the stack. It uses active firewalls, validation protocols, authentication states (Sessions / JWT), and Web Application Firewalls (WAF) to defend against malicious traffic.

Typical Request Flow:

Browser → DNS / IP Lookup → Load Balancer / Reverse Proxy → Web Server Core → Application Layer → Data Base → Response Back to Browser.

3. Popular Web Server Platforms

- **Apache HTTP Server**: An open-source, highly customizable, stable, and platform-independent system widely adopted across global infrastructure.
- **NGINX**: A light, event-driven web server focused on processing high numbers of concurrent requests. It is commonly configured as an edge reverse proxy and load balancer.
- **Microsoft IIS (Internet Information Services)**: A proprietary extensible server framework designed by Microsoft to run smoothly within Windows OS platforms and tightly link with the .NET runtime environment.
- **LiteSpeed**: A fast commercial server solution designed as a direct drop-in replacement for Apache configurations, delivering rapid resource handling.
- **Apache Tomcat**: A specialized environment engineered to execute Java Server Pages (JSP) and Java Servlets.

- **Caddy:** A modern, production-ready server featuring automatic HTTPS out of the box with straightforward, human-readable configuration parameters.

Platform Comparison: Microsoft IIS vs. Apache

Feature	IIS (Internet Information Services)	Apache HTTP Server
Developer	Microsoft Corporation	Apache Software Foundation
Platform Support	Windows Operating Systems exclusively	Cross-platform (Linux, Windows, macOS, etc.)
License Model	Proprietary / Commercial	Open Source (Free Software)
Configuration	Graphical User Interface (GUI Management Console)	Plaintext configuration files (e.g., <code>.htaccess</code> , <code>httpd.conf</code>)
Integration Ecosystem	Optimized out-of-the-box for Microsoft .NET	Highly flexible; excels with PHP, Perl, Python, etc.
Financial Cost	Requires a valid Windows licensing model	Completely free

4. Types of HTTP Requests

HTTP specifies standard request methodologies that determine the actions a client browser intends to execute on server-side resources:

1. **GET:** Retrieves information from the server. Target variables and key-value payload fields are sent appended directly inside the URL query string.
Use Cases: Loading static pages, performing searches, reading a specific product listing.
2. **POST:** Submits structured data to the server inside the request body payload to create a new resource or process an entity.
Use Cases: Registering new accounts, processing checkouts, uploading files.
3. **PUT:** Replaces or updates an entire target resource with a new payload provided inside the request body.
Use Cases: Overwriting an existing profile file with entirely new contents.
4. **PATCH:** Partially modifies an existing server resource, changing only the specific properties defined in the request body.
Use Cases: Updating a single user parameter, like changing an email address or password.
5. **DELETE:** Completely removes a specified resource from the backend server storage.
Use Cases: Deleting a row or an unneeded user account from a database.

6. **HEAD:** Functions exactly like a `GET` request, but instructs the server to omit the body content from the response, returning only the headers.

Use Cases: Reviewing file size metrics, checking resource modifications, or confirming link validity.

7. **OPTIONS:** Requests a list of the HTTP methods and communication options currently allowed and supported by the target server configuration.

Data Transmission Comparison: `$_GET` vs. `$_POST`

Feature	GET Method	POST Method
Visibility	Data parameters are appended explicitly within the visible browser URL string.	Data is encapsulated securely inside the hidden HTTP request body.
Length Limits	Strictly limited by browser and server URL length restrictions (typically ~2KB).	No rigid technical length constraints; accommodates large data payloads.
Use Case Suitability	Best for safe, non-sensitive operations like text queries and data sorting.	Mandatory for secure forms, user passwords, and file uploads.
Caching Support	Can be completely cached and logged within browser histories.	Never cached or saved by default browser history mechanics.
Bookmarking	URL states can be cleanly bookmarked and shared with others.	Cannot be bookmarked to preserve state execution.

5. Introduction to PHP (Hypertext Preprocessor)

PHP is an open-source, server-side scripting language explicitly developed to produce dynamic, data-driven web pages.

Key Benefits & Features:

- **Open Source:** Completely free to use, modify, and deploy globally without licensing overheads.
- **Server-Side Scripting:** The logical instructions run entirely on the hosting web server. The runtime environment interprets code blocks, compiles them on the fly, and yields a clean, static HTML output stream to the browser.
- **Easy HTML Integration:** Code blocks can be embedded directly alongside static HTML text using standard `<?php ... ?>` tags.
- **Robust Data Connectivity:** Integrates naturally with relational databases like MySQL to execute fast storage, querying, and updating functions.
- **Platform Independence:** Runs consistently across almost all major operating systems, including Linux, Windows, UNIX, and macOS.

Example: Basic "Hello World" Script

```
<!DOCTYPE html>
<html>
<head>
  <title>Hello World Page</title>
</head>
<body>
  <?php
    echo "Hello world";
  ?>
</body>
</html>
```

Expected Browser Output: Hello world

Accessing PHP Files on a Web Server:

1. Write your script and save it with a valid file name ending in `.php` (e.g., `index.php`).
2. Place the file inside the root document directory of your local web server installation (e.g., the `htdocs` folder in an Apache XAMPP stack).
3. Ensure your web server engine is active and running.
4. Open your browser and navigate to the address: `http://localhost/index.php`.

6. PHP Fundamentals: Language Constructs

Core Data Types

- **Integer:** Whole numbers without decimal places.
`$ch = 11;`
- **Float / Double:** Fractional numbers containing a floating decimal point.
`$num = 11.33;`
- **String:** Ordered sequences of characters bounded by quotes.
`$name = "Nandini";`
- **Boolean:** Evaluates exclusively to either `true` or `false`.
`$islogin = true;`
- **Array:** A structured variable holding multiple distinct values keyed by index or name.
`$colors = array("Red", "Green", "Blue");`
- **Object:** A live instance of a class, enabling object-oriented program architectures.

```
class Students {  
    public $name;  
}  
$obj = new Students();
```

- **Null:** A unique data state denoting a variable explicitly declared without any assigned value.

```
$ch = null;
```

Variable Rules & Usage

- Must always start with the leading dollar symbol (\$).
- The first character after the dollar sign must be either a letter or an underscore (_). It cannot begin with a numerical value.
- Variables are strictly case-sensitive (e.g., \$name and \$NAME are treated as separate variables).

```
<?php  
$name = "Nandini";  
$age = 18;  
  
echo "Name: " . $name . "<br>";  
echo "Age: " . $age;  
?>
```

Output:

Name: Nandini

Age: 18

Constants

Constants represent fixed configuration values that cannot be modified or redefined anywhere across the execution scope after initial instantiation. They are declared explicitly using the global `define()` function construct, and they do not use a leading \$ character.

```
<?php  
define("CH", "Nandini");  
echo "Name: " . CH;  
?>
```

Output: Name: Nandini

7. String Manipulation Functions

- `strlen($str)` : Counts the total characters in a string.

Example: `strlen("helloworld");` → 10

- `str_word_count($str)` : Counts words separated by spaces.
Example: `str_word_count("hello world");` → **2**
- `strrev($str)` : Reverses character order.
Example: `strrev("Nandini");` → **inidnan**
- `strtoupper($str)` : Converts text to all-uppercase letters.
Example: `strtoupper("reshma");` → **RESHMA**
- `strtolower($str)` : Converts text to all-lowercase letters.
Example: `strtolower("NaNDini");` → **nandini**
- `str_replace($search, $replace, $subject)` : Substitutes all occurrences of a search string with a replacement string.
Example: `str_replace("Nandini", "Mandhu", "Hi Nandini");` → **Hi Mandhu**
- `strpos($haystack, $needle)` : Locates the index position of the first occurrence of a substring.
Example: `strpos("Hello Mandhu", "Hello");` → **0**
- `trim($str)` : Strips whitespace characters from both ends of a string.
Example: `trim(" hello ");` → **"hello"**

8. Working with Arrays

Arrays bundle multiple sequential data items together within a unified container name. Element tracking lengths are found using the `count()` utility function, and arrays can be indexed sequentially using a standard `for` loop.

```
<!DOCTYPE html>
<html>
<head>
    <title>PHP Arrays Example</title>
</head>
<body>
    <h2>PHP Array Example</h2>
    <?php
        $colors = array("Red", "Green", "Blue", "Yellow");
        echo "Array elements:<br>";
        for ($i = 0; $i < count($colors); $i++) {
            echo $colors[$i] . "<br>";
        }
    ?>
</body>
</html>
```

9. Processing HTML User Forms

Forms capture client data entries and transmit them to the server layer. When using `method="post"`, the parameters are passed securely inside the HTTP request body. On the backend, PHP extracts these parameters using the superglobal array `$_POST['input_name']`.

```
<!DOCTYPE html>
<html>
<head>
    <title>PHP Form Examples</title>
</head>
<body>
    <h2>Student Registration Form</h2>
    <form method="post">
        Name: <input type="text" name="name"><br><br>
        Age: <input type="number" name="age"><br><br>
        Course:
        <select name="course">
            <option>CSE</option>
            <option>ECE</option>
            <option>MECH</option>
        </select><br><br>
        <input type="submit" name="submit" value="Submit">
    </form>

    <?php
        if (isset($_POST['submit'])) {
            $name = $_POST['name'];
            $age = $_POST['age'];
            $course = $_POST['course'];

            echo "<h3>Student Details</h3>";
            echo "Name: " . $name . "<br>";
            echo "Age: " . $age . "<br>";
            echo "Course: " . $course;
        }
    ?>
</body>
</html>
```

Example Processing Flow Output (e.g., executing form via localhost/form.php):

`Student Details`

Name: chodi

Age: 23

Course: CSE

10. State Management: Cookies vs. Sessions

Because the HTTP protocol layer is natively stateless, web applications rely on cookies and sessions to maintain persistent user states across sequential interactions.

Cookies

A cookie is a lightweight data text file sent from a web server and stored locally on the user's web browser client. They are commonly used for basic session management (tracking active log-ins), interface personalization (retaining visual themes or language preferences), and visitor analytics tracking.

Lifecycle: Session-scoped cookies disappear automatically when the browser is closed, whereas persistent cookies stay active until they hit an explicit expiration Unix timestamp. They can be created in PHP using the `setcookie(name, value, expire, path)` function construct.

Sessions

A session forms a temporary, interactive, secure information connection linking a unique client user with the server environment across multiple requests.

Lifecycle: Sessions are assigned a unique, system-generated **Session ID** token string, which is typically stored in a browser cookie or passed via hidden form properties. The actual data payload rests securely inside server-side storage away from user tampering, making them highly secure for authorization checkpoints and sensitive shopping cart parameters.

Feature Comparison: Sessions vs. Cookies

Feature	Sessions	Cookies
Storage Location	Stored securely on the Web Server	Stored locally on the client's Web Browser
Data Size Limit	Supports substantial storage capacities	Strictly limited to a maximum of 4KB per cookie file
Data Lifetime	Temporary (Expires after periods of inactivity)	Persistent (Can remain active for months or years)
Security Tier	Highly secure (Protected from external tampering)	Lower security (Vulnerable to interception/modification)
Accessibility	Accessible only by server-side execution code	Accessible by both the server and local JavaScript
Primary Application	Managing login authorizations and checkout states	Storing interface themes and analytics tracking identifiers